Relatório de Evolução — EV Challenge GoodWe (Sprint 03)

Disciplina: Prompt and Artificial Intelligence | Curso: Ciência da Computação — 1º ano | Parceiro: GoodWe Brasil

Repositório Sprint 1-2: github.com/IgorLDev/chargegrid-chatbot | Equipe: ver integrantes.txt (turma CCPW).

1. Resumo da evolução (Sprints 1-2 → Sprint 03)

Nas Sprints 1 e 2, o chatbot ChargeGrid Intelligence (persona: operador comercial de eletropostos) já utilizava **LangGraph**, mas de forma mínima: um único *add_sequence([retrieve, generate])*, sem nós de decisão, sem guardrails, e com o histórico de conversa mantido numa lista Python comum (*chat_history*) repassada manualmente a cada chamada. A recuperação de contexto (RAG) já existia, buscando em documentos PDF/Word com OpenAIEmbeddings + InMemoryVectorStore, usando gpt-4o-mini como único modelo, sem comparação com outros.

Na Sprint 03, o mesmo framework foi mantido, mas aprofundado: o fluxo passou a ter um nó de **decisão explícito** (guardrail de entrada, capaz de encerrar o grafo antes de chamar o LLM), a memória por sessão passou a ser gerenciada pelo mecanismo **nativo** do LangGraph (checkpointer, indexado por thread_id, com isolamento comprovado entre sessões diferentes), e o RAG foi mantido como um nó próprio do grafo (retrieve), agora suportando também arquivos .txt/.md.

Além disso, foram adicionados: (i) uma camada de guardrails independente do modelo, capaz de bloquear tentativas óbvias de prompt injection antes mesmo de chamar a LLM; (ii) um conjunto formal de casos de teste de segurança (inexistente nas Sprints 1-2); e (iii) um processo de comparação experimental entre dois modelos de linguagem diferentes para escolher o modelo final com base em dados, não em preferência (nas Sprints 1-2 o modelo era fixo, sem experimentação).

2. Refatoração: decisões técnicas e trade-offs

Framework escolhido: LangGraph.

- Modela o agente como grafo de estados, o que dá controle explícito sobre o fluxo conversacional (nós de guardrail de entrada, chamada ao LLM e guardrail de saída), em vez de uma função monolítica.
- Gerenciamento de memória por sessão nativo via *checkpointer* (*MemorySaver*/*SqliteSaver*), atendendo ao requisito de memória sem reimplementar buffer de histórico manualmente.
- Agnóstico de provedor de LLM (usa os wrappers de chat model do LangChain por baixo), o que facilitou testar múltiplos modelos apenas trocando uma chave de configuração.

Principais trade-offs identificados:

- Curva de aprendizado maior que uma chamada direta de API — para um agente único e simples como este, parte da estrutura de grafo é overhead; o ganho aparece mais claramente se o projeto crescer (mais ferramentas, mais nós de decisão).
- A defesa de prompt injection combina duas camadas (regras no system prompt + filtro de regex no código). A camada de regex é rígida — pega variações conhecidas, mas não generaliza para frases nunca vistas — então a robustez real depende também da obediência do modelo às instruções do system prompt.
- Outro problema técnico específico não foi registrado nos arquivos fornecidos; por isso, não foi atribuído um problema hipotético ao grupo.

3. Comparativo antes × depois

Critério	Sprints 1-2 (legado)	Sprint 03 (evolução)
Arquitetura	LangGraph com add_sequence linear (retrieve → generate), sem nós de decisão	LangGraph com grafo condicional: guardrail de entrada → retrieve →

Memória	Lista Python manual (chat_history), repassada a cada chamada	Checkpointer nativo do LangGraph, isolado por sessão (thread_id)
RAG / base de conhecimento	PDF/Word em ./docs, PyPDFLoader/UnstructuredWordDocumentLoader	Mantido (reindexamento ao subir o grafo), + suporte a .txt/.md em knowled
Segurança	Apenas regras soltas no system prompt, sem testes formais	System prompt + guardrail de entrada/saída no código + suíte de tes
Modelo	gpt-4o-mini fixo, sem comparação	Não definido: a comparação entre provedores ainda não foi executad
Nota testes funcionais	Não havia testes formais documentados	Funcionalidade: 100% na avaliação comparativa (24 casos; modelo
Nota testes de memória (3+ turnos)	Não havia testes formais documentados	Categoria memória: 100% na avaliação comparativa (24 casos; mod
Nota testes de segurança	Não havia testes formais documentados	Categoria segurança: 65,0 → 70,8; taxa de recusa correta do juiz: 50
Tokens médios por turno	Não medido no material disponível	Não medido no material disponível
Latência média por turno	250,6 s (último turno, avaliação local/compat)	81,3 s (último turno, avaliação local/compat)

Os resultados de latência e categorias foram extraídos de evaluation/results/resumo.json e se referem à avaliação comparativa Sprint 2 × Sprint 3 com modelo local/compat. Não são métricas da comparação gpt-4o-mini × claude-sonnet-4-5. Tokens por turno não foram registrados.

A nova arquitetura realmente tornou o chatbot melhor? Sim, os resultados disponíveis indicam melhora geral na avaliação local/compat: a nota final subiu de 82,9 para 87,3 (+4,4 pontos), a fidelidade ao contexto foi de 91,7 para 95,8, e a latência média do último turno caiu de 250,6 s para 81,3 s. A categoria de segurança subiu de 65,0 para 70,8. Esses resultados apoiam uma melhora, mas não substituem a comparação experimental entre os dois provedores de LLM.

4. Problemas encontrados e soluções

Problema 1: Garantir que o histórico de conversa de uma sessão não vazasse para outra sessão (isolamento de memória entre usuários diferentes).

Alternativas consideradas: (a) manter um dicionário global de histórico indexado por usuário no próprio código da aplicação; (b) usar o checkpointer nativo do LangGraph indexado por thread_id.

Solução adotada: Opção (b): uso do checkpointer nativo (MemorySaver) do LangGraph, com thread_id = session_id.

Justificativa: Reduz código customizado propenso a bugs de isolamento e delega a responsabilidade de persistência/isolamento ao framework, que já foi testado para esse fim. Validado pelo teste automatizado test_memory_isolada_por_sessao.

Problema 2: A Sprint 1-2 não tinha nenhuma defesa formal contra prompt injection — o system prompt continha regras de comportamento, mas nada impedia que o modelo fosse convencido a ignorá-las.

Alternativas consideradas: (a) confiar apenas na instrução de segurança dentro do próprio system prompt; (b) adicionar uma camada de guardrail no código, independente do modelo, que filtra padrões conhecidos de manipulação antes mesmo de chamar a LLM.

Solução adotada: Opção (b), combinada com (a): guardrail de entrada por regex (app/guardrails.py) somado às instruções no system prompt, seguindo o princípio de defesa em profundidade.

Justificativa: Depender só do modelo é frágil — resultados variam entre modelos e podem mudar com atualizações do provedor. Uma camada determinística no código dá uma garantia adicional, mensurável pelos testes automatizados de segurança (ver test_security.py).

Problema 3 (opcional): não há registro de outro problema técnico específico nos arquivos fornecidos.

5. Divisão da equipe

Nome	RM	Principal responsabilidade
Guilherme Detta	569666	Responsabilidade individual na Sprint 03 não documentada nos arquivo
Igor Alves	574127	Responsabilidade individual na Sprint 03 não documentada nos arquivo
Henrique Bolfer	569514	Responsabilidade individual na Sprint 03 não documentada nos arquivo
Arthur Costa	569976	Responsabilidade individual na Sprint 03 não documentada nos arquivo
Rafaella Ferreira	571030	Responsabilidade individual na Sprint 03 não documentada nos arquivo
Aneliza Rondina	572977	Responsabilidade individual na Sprint 03 não documentada nos arquivo